

You Only Look Once: Unified, Real-Time Object Detection

📅 작성일시	@2025년 3월 26일
📄 강의 번호	BASE
📄 유형	논문리뷰
📎 자료	https://arxiv.org/pdf/1506.02640
☑️ 복습	☐



1. 기존 검출 기법처럼 후보영역을 따로 생성하지 않고, 이미지 한 장을 한 번에 보고 바운딩 박스와 클래스 확률을 직접 회귀함
2. 이렇게 통합된 단일 CNN 구조는 별도의 파이프라인 없이 end-to-end로 학습할 수 있어 단순하며 최적화가 용이함.
3. 기본 YOLO는 GPU에서 초당 약 45프레임, Fast YOLO는 155프레임 까지 달성해 사실 상 실시간 검출이 가능함
4. 작은 물체 탐지나 정교한 위치 추정에 약점이 있지만, 배경을 잘못 인식하는 오류는 훨씬 적음
5. 실제 영상(웹캠)이나 도메인이 다른 이미지에도 비교적 높은 일반화 성능을 보여 실시간 응용에 매우 적합

ABSTRACT

1) 기존의 객체 검출 접근

- 이전 연구들은 주로 분류 네트워크를 “후보영역 마다 여러 번” 적용하는 방식으로 객체 검출을 수행함
- 예: Selective Search 같은 기법으로 여러 박스를 뽑고(Region Proposal), 그 안을 CNN으로 분류 → 매우 느리고 복잡

2) YOLO의 새로운 접근

- YOLO는 “객체 검출”을 이미지 전체를 입력으로 받아 바로 좌표(bounding box)와 클래스 확률을 회귀(Regression)하는 문제로 간주.
- 즉, 이미지 전체를 한 번(One Look)만 보고, 동시에 여러 박스와 클래스 확률을 예측

3) 단일 신경망(end-to-end)구조

- 파이프라인을 여러 단계로 나누지 않고, 하나의 합성곱 신경망(Convolutional Neural Network)으로 통합
- 이 통합 모델을 직접 검출 성능을 최대화 하는 방향으로 학습할 수 있음

4) 속도와 정확도

- 기본 YOLO모델은 GPU에서 초당 45프레임으로 동작하며, FAST YOLO는 155FPS까지 도달

- 다른 실시간 모델에 비해 mAP 2배 이상
- 다만 객체의 정확한 위치 추정 (특히 작은 객체)에 있어 약간의 오차가 있음.
- 배경(Background)을 객체로 잘못 인식하는 오류는 다른 방법보다 훨씬 적음
- 새로운 도메인(그림, 예술 작품)에도 잘 일반화 됨

4) 결론

- 빠른 속도와 단순함, 그리고 비교적 높은 정확도를 모두 갖춘 객체 검출 모델
- 다른 복잡한 단계없이, 한 번의 CNN추론으로 객체 위치와 클래스 확률까지 동시에 예측

INTRODUCTION

1) 동기

- 사람은 이미지를 빠르게 보고 순간적으로 "무엇이 어디에 있는지" 파악할 수 있음.
- 컴퓨터도 비슷한 속도와 정확도를 가진다면, 자율주행이나 AR/VR, 로봇 비전 같은 분야에서 큰 혁신 가능

2) 기존 방식의 한계

- 흔히 써 온 슬라이딩 윈도우 나 Region Proposal(R-CNN계열)접근은
 - 이미지를 여러 위치/크기로 잘라서 CNN등으로 검사하거나
 - Selective Search 등으로 후보영역을 만들고 → 각각을 분류 → 후처리 (NMS)
 - 여러 단계를 거쳐야 하고 각 단계를 따로 최적화 해야 하므로 복잡하고 느림

3) YOLO: 단일 네트워크로 통합

- YOLO는 이미지를 SxS격자로 나눈 뒤, 각 격자 셀이 2개의 박스(B개의 박스 일반화 가능)와 그 박스에 대한 클래스 확률을 예측
- 따라서, 하나의 CNN이 전체 이미지를 보고 바운딩 박스와 클래스를 한 번에 내놓음.
- 파이프라인을 쪼개지 않고, end-to-end로 학습 가능함

4) 장점

- 매우 빠름: CNN을 한 번만 추론하기 때문에 실시간(초당 수십~수백 프레임) 처리 가능.
- 글로벌 문맥 정보 활용 : 이미지를 전체적으로 보기 때문에, 배경을 객체로 착각하는 일이 상대적으로 적음
- 일반화 성능 우수: 자연 이미지에서 학습해도, 예술 작품 등 다른 도메인으로도 꽤 잘 적용 됨.

5) 한계

- 작은 객체나 서로 근접해 있는 객체들은 정확한 위치를 잡아내기 어려울 수 있음 (격자 구조의 한계)
- 위치(localization)자체를 예측하는 것이므로, 작은 바운딩 박스에는 오차가 크게 반영되는 편
- 그래도 전체 속도 대비 검출 성능이 매우 좋아, 실시간 객체 검출에 특화된 강력한 방법론

UNIFIED DETECTION

1)YOLO: 단일 신경망을 통한 "통합된" 객체 검출

- 아이디어
 - 기존처럼 "후보영역 → 분류"라는 분리된 파이프라인이 아니라, 이미지를 바로 입력으로 받아 바운딩박스 좌표 + 클래스 확률을 동시에 회귀하는 방식
 - 이미지 전체를 $S \times S$ 격자로 나눈 뒤, 각 격자 셀이 예측해야 할 정보:
 - B개의 바운딩 박스 (각 4개의 좌표 + 1개의 confidence = 5개 값)
 - C개의 클래스 확률 (조건부 확률 $P(\text{Class}|\text{Object})$)
 - 결국, $(B \times 5 + C)$ 개의 값을 $S \times S$ 격자마다 예측 → 최종 출력은 $(S \times S \times (B \times 5 + C))$ 텐서
- Confidence와 클래스 확률 계산
 - confidence 점수는 "해당 바운딩박스가 물체를 실제로 포함할 가능성 x 정확도"를 의미.
 - 구체적으로, IoU를 이용해서 예측박스와 실제 정답박스가 얼마나 겹치는 지를 학습
 - 최종적으로 $(P(\text{Class}) \times \text{IoU})$ 가 클래스별 점수가 되어, 객체 클래스와 위치를 동시에 얻을 수 있음
- 네트워크 구조
 - 논문에서는 GoogLeNet 스타일을 참고한 합성곱 신경망을 사용하되, Inception 모듈 대신 1×1 conv + 3×3 conv 조합을 반복
 - 입력 이미지는 448×448 로 리사이즈해서 처리
 - 최종 fully-connected 층에서 $S \times S \times (B \times 5 + C)$ 를 출력
- 학습 방식
 - Imagenet분류 (224×224 입력)로 먼저 합성곱 계층을 사전학습(Pretrain)후 검출 용으로 구조와 입력 크기를 확장(448×448)
 - Loss 함수 : 예측된 박스 좌표/크기, confidence, 클래스 확률 간의 오차를 모두 합산
 - localization(좌표) 오차는 가중치를 크게, 객체가 없는 박스에 대한 confidence 오차는 작게 줘서 학습 안정
 - 한 셀에서 B개의 박스를 낼 때, 실제와 가장 겹치는 박스만 책임지도록 해서 각 박스가 역할을 분담하도록 함
- 테스트 시(추론)
 - 이미지 1장을 CNN에 통과 → $S \times S \times (B \times 5 + C)$ 텐서 → confidence와 클래스 확률로 필요시 NMS(Non-Max Suppression)처리
 - 단 한 번의 CNN 추론으로 모든 바운딩 박스와 클래스가 나옴 → 따라서 매우 빠름.
- 한계
 - 격자 한 셀이 한 번에 여러 객체(특히 작은 객체나 근접 객체)를 정확히 처리하기 어려울 수 있음
 - 작은 물체를 다루는 데 정확도가 다소 떨어짐 (좌표 오차가 크게 반영될 수 있음)
 - 그럼에도 불구하고, 속도와 간결함에서 큰 장점을 제공

Comparison to Other Detection Systems

YOLO와 대표적인 객체 검출 시스템들을 비교할 때의 특징들을 요약해 보면 다음과 같음.

1) Deformable Parts Model(DPM)

- 예전부터 유명한 전형적인 슬라이딩 윈도우 기반 모델
- 이미지 곳곳을 일정 간격으로 분할하고, HOG 등 전통 피쳐 + SVM 등을 이용해 “물체인지?”를 검사
- 단계별로 분리된 파이프라인 + 느린 속도 + 정확도에서 현대 CNN 기반 대비 낮음.
- 반면 YOLO는 단일 CNN으로 분리된 모듈 없이 end-to-end 처리

2) R-CNN 계열 (Selective Search , Fast R-CNN, Faster R-CNN 등)

- R-CNN
 - Selective Search 로 후보 영역 (약 2천개)을 추출 → 각 영역을 CNN으로 분류 → 후처리
 - 매우 느리고(이미지당 수십 초), 단계별로 따로 학습해야함
- Fast R-CNN
 - 후보 영역을 공동 합성곱으로 추출 + ROI Pooling + 클래스 분류 등으로 속도를 개선
 - 그래도 후보 영역 생성에 Selective Search 가 필요 → 여전히 느림
- Faster R-CNN
 - Selective Search 대신 “Region Proposal Network (RPN)”을 이용 → GPU에서 후보 영역도 CNN으로 예측
 - 속도가 상당히 빨라졌으나, 여전히 7~18fps 정도
- 반면, YOLO는 후보 영역이라는 개념 없이, 격자 단위로 bounding box를 직접 회귀함
 - 단일 CNN으로 모든 걸 처리 → 한 번 추론으로 결과 산출 → 속도 우수

3) OverFeat

- 이미지 분류 + 위치 추정을 CNN으로 같이 해보려 했던 과거 연구
- 하지만 여전히 슬라이딩 윈도우(다양한 스케일)를 기반으로 여러 위치를 탐색 → 후처리(NMS) 필요
- YOLO 는 격자 구조를 통해 훨씬 단순한 방식으로, 모든 위치/크기를 한 번에 처리

4) MultiBox/ Deep MultiBox

- CNN으로 “바운딩박스 후보” 만 뽑아주는 proposal 방식
- 여전히 후보영역 뒤에 다른 분류 단계가 붙어야 하므로, 완전한 검출 파이프 라인은 아님
- YOLO는 객체 검출 파이프라인 자체가 하나의 CNN으로 끝남

5) 속도 비교

- DPM, R-CNN 등은 수 초 이상 소요
- Fast R-CNN, Faster R-CNN 은 많이 빨라졌지만 수 fps ~ 십수 fps 수준
- YOLO는 GPU 기준 45fps (기본 YOLO), 155fps(Fast YOLO)달성 → 실시간 처리 가능

Experiments

(1) 실험 세팅

- 데이터셋: 주로 PASCAL VOC 2007, VOC 2012를 사용해 mAP(mean Average Precision) 측정.

- **비교 대상:** R-CNN 계열(Fast/Faster R-CNN), DPM, OverFeat 등 다양한 객체 검출 방법과 성능 및 속도 비교.

(2) PASCAL VOC 2007 결과

- **YOLO vs. 기존 방법**
 - **Fast YOLO: 155 fps / 약 52~53% mAP** (당시 기준 가장 빠른 실시간 검출).
 - **기본 YOLO: 45 fps / 약 63% mAP.**
 - Faster R-CNN(ZF)는 18 fps 정도, VGG 모델은 더 정확하지만 속도는 7 fps.
 - 즉, YOLO는 속도가 월등히 빠르면서, 기존 실시간 모델 대비 정확도도 2배 이상 높음.
- **오류 분석**
 - YOLO는 **배경을 오인식(Background False Positives)**하는 실수가 적음.
 - 다만 **작은 물체나 위치(localization) 정교함**에서 다소 취약: 예컨대 바운딩박스가 정확히 물체를 둘러싸지 못하는 경우가 비교군 대비 많음.
- **결과 결합(Ensemble) 효과**
 - Fast R-CNN과 YOLO의 검출 결과를 결합(서로 상호보완)하면 정확도가 크게 상승(mAP +3%p).

(3) PASCAL VOC 2012 결과

- YOLO 단독으로 약 **58% mAP** 달성 (Fast/Faster R-CNN 등 최신 고정밀 기법보다는 낮음).
- 여전히 배경 구역을 잘못 검출하는 일은 드물지만, 작은 물체 탐지나 정확한 경계 예측에서는 뒤처짐.
- Fast R-CNN과 YOLO를 합치면 상위권 모델과 견줄 만한 성능(약 70%+ mAP) 확보.

(4) 일반화 성능 (Artworks 등)

- 그림(Artwork) 같은 도메인에 모델을 적용했을 때, **R-CNN** 등은 Selective Search가 일반화에 취약해 성능이 급락.
- 반면 YOLO는 **사이즈/위치 등 전체 맥락적 특성**을 함께 학습하기 때문에, 도메인 달라져도 상대적으로 성능 하락이 작음(즉, 잘 일반화).

Real-Time Detection In The Wild

1. 실시간 웹캠 데모

- YOLO를 웹캠 영상에 직접 연결 → CNN 한 번 추론으로 매 프레임 객체 검출.
- 45fps(기본 YOLO)나 150fps(Fast YOLO)로 동작 가능하므로, ****거의 즉시성(interactive)****이 가능.

2. 실용성

- 로봇 비전, 자율주행, AR/VR, 감시 시스템 등, **실시간**이 중요한 분야에 손쉽게 적용할 수 있음.

Conclusion

1. 주요 기여

- 객체 검출을 **'이미지 → (박스 + 클래스)'**를 단 한번에 추론하는 하나의 **CNN**으로 재정의.
- 파이프라인을 단순화하고, **end-to-end**로 학습.
- 이 접근 덕에 ****실시간 처리 속도(수십~수백 fps)****와 간결한 구조를 동시에 달성.

2. 장점 요약

- **매우 빠른 속도:** 슬라이딩 윈도우, 후보영역 탐색 등 없이 단일 CNN만으로 검출 수행.
- **글로벌 문맥 처리:** 전체 이미지를 한눈에 보며 학습 → 배경 영역을 잘못 인식하는 일이 적음.
- **일반화 능력:** 도메인(예: 예술 작품) 바뀌어도 기존 방법보다 성능 하락이 상대적으로 작음.

3. 한계와 미래 과제

- 작은 객체나 서로 근접한 물체들을 정밀하게 잡아내는 데는 상대적으로 약함.
- 이후 Mask R-CNN, YOLOv2/YOLOv3 등 후속 연구에서 이러한 한계들이 점차 보완됨.